Multilingual University Email Triage and Classification

Hesam Mohebi

7505478@studenti.unige.it

10/06/2026

1 Details of the proposal

1.1 Full specification of your proposal

This project proposes a multilingual email classification for the emails received from UniGe services on topic and priority. The system is based on real in-box emails extracted and annotated using LLMs with further manual corrections. After preprocessing, ablation studies were performed between TF-IDF baselines, hybrid featured-based models, and transformer fine-tuning. The goal is to aid students being notified for urgent emails based on their topics.

1.2 The kind of proposal

Creative
https://github.com/hehesam/NLP-project#data-folder

1.3 The range of points/difficulty of proposal

Considering working with real data and annotating it, ablation studies, and live demo it is a **Hard** project.

2 Introduction

2.1 Motivation

A student in UniGe, is receiving a constant stream of emails daily. These emails originate from different sources like Aulaweb, Administrative offices, subscribed platforms, personal emails from professors or peers, and advertisement. Furthermore, since webmail service is independent from mainstream mailing platforms such as Gmail and Outlook, the repetitive process of login-in becomes an additional effort and as a result, disincentivize students to constantly check their emails. Finally, receiving Italian emails among English ones is another issue specifically for international students whom lack the proficiency in Italian, therefore a student like myself can easily miss an important email among multiple Italian emails.
The motivation behind this project is to build a system that reduces the overhead and increases availability. In order to reach this goal, a naïve keyword filtering is too brittle to handle emails' variability, also a full neural pipeline is too heavy for the scale of dataset. Therefore, classical NLP seems a feasible approach.

2.2 Project Goal

The objectives of this project are to perform two independent tasks deliberately not turning into a single multi-label model:

- Classifying emails' topics: it answers the question of what is this email about? It is a lexical task which leads to one of the five classes below:
    - Administrative: institutional services, offices, accounts, elections, and official procedures.
    - Course-exam: lectures, exam dates, course platforms, grading, lab activity.
    - Event: Seminars, conferences, talks, public events, ceremonies.
    - Deadline-action: explicit deadlines or required student actions.
    - Advertisement: commercial and promotional content.
- Prioritizing emails: it answers the question of how urgent is this email? (High, Medium, Low). Priority is pragmatic and contextual; the same lexical can be high or low depending on the data, sender, or it is announcement or routine reminder.

2.3 Contribution

This project makes five main contributions:

- A real multilingual university email dataset: instead of using a pre-existing benchmark, the corpus was extracted from UniGe mailbox.
- A two-task formulation: This separation is critical since the topics is mostly lexical, while priority depends more on context, urgency, and student relevance.

- LLM assisted annotation with manual cleaning: each email was labeled independently by multiple LLMs, and disagreement cases were manually reviewed using models' rationales and confidence scores.
- Comparative modeling pipeline: I conducted a systematic comparison of different NLP approaches from TF-IDF baselines, then hybrid features such as metadata, regex patterns, etc. and finally comparing these models with fine-tuned transformer.
- MPV application: implementing a Telegram bot demonstrates the practicality of this system for real usage.

3 Dataset Construction

3.1 Email Extraction

To extract the emails from mailbox, I implemented a four-step pipeline script that turns a live IMAP mailbox into a local, tabular corpus:

1) Connection and Authentication: pens a TLS connection to the university IMAP server, then logs in with credentials.
2) Select and filter mail: opens a mailbox (default inbox, or another folder), runs an IMAP search.
3) Normalize each message: parses the raw emails and decodes the header [1].
4) Emits a tabular corpus as JSONL + CSV.

3.2 Dataset Characteristics

The annotated dataset roughly 1000 emails after extraction and ~920 after cleaning. Approximately 58% of the corpus is Italian and 39% is English. The dataset itself consist of:

- Metadata: (from, from_domain, to, date_iso, has_attachment).
- Text fields: (subject, body_plain).

4 Annotation Methodology

Hand-labelling 1000 emails alone is time-consuming and inefficient. Therefore, using LLMs can drastically speed-up the processes yet it is prone to hallucination and systematic biases. As a result, I came up with three-LLM annotation strategy.

4.1 Label Schema

The label schema was created based on the practical need of classifiers. Each email is annotated along two main dimensions: topic and priority. Topic answers what the email is mainly about, while priority classifies how urgently the student should read or act on it. The two tasks are kept separate because each describe different aspects of email. Topic is mostly

based on lexical and semantic content whereas priority depends heavily on context, deadline proximity and senders' importance.

Because many university emails contain multiple intentions, the schema also included decision rules for ambiguous cases. Labels are assigned to dominant purpose of the email rather than isolated keywords. For example, an event is labelled as deadline-action only if the main purpose was to make the student register. Another example is when an administrative notice was labelled deadline-action only when the required action is more important than the administrative information itself. When the subject and body suggested different labels, the body is given more weight because it contains the actual intention of email.

4.2 LLM-Assisted Annotation

The three-LLM annotation strategy consists of several steps. First, in order to maximize the context window usage and increase efficiency, emails are passed in 19 baches with JSONL format. Second, along with the batches, an identical master prompt is also attached, this way, we ensure that all of the LLMs follow the same set of instructions even though their internal reasoning and model bias could differ.

The master prompt itself is a central part of the annotation design. It determines the full label schema, including topic labels, priority labels, confidence scores, rationales, and privacy-related fields. It also defines decision rules for ambiguous cases such as when an email is classified as deadline-action rather than event. This is crucial since many emails contain overlapping intentions. For example, an email announcing an event but also the student should register before a deadline.

In the third step, each LLM is required to provide a short rational and a confidence score for its predictions. This way, the annotation process becomes more transparent while the confidence score helps identifying ambiguous cases. These two features were especially helpful in cleaning stage, because emails with low confidence or agreement can be manually inspected using model's explanations rather than just their label.

4.3 Agreement and Cleaning

After all emails were annotated using LLMs, the downstream cleaning pipeline classifies each email into one of three categories in table 1:

Agreement	Frequency	Strategy
3/3 agreement	68.1%	Accept the shared label as high confidence
2/3 agreement	29.8%	Use the majority label and log the disagreement for review
1/3 agreement	2.1%	Manual adjudicate

Table 1: Model Agreement Distribution

The first stage of cleaning removes unreliable rows, including empty records, duplicated identifiers, invalid IDs, and annotations with insufficient completeness. In the end, a filter checks whether each model annotation is fully complete or not.

After the initial cleaning I inspected the distribution of labels produced by Gemini Pro 3.1, Claude Haiku 4.5, and GPT 5 thinking. The original proposal included six topic classes yet, through the initial observations a consequential change was made to remove class Other. This decision was based on several reasons. First, many emails labelled as Other could still be assigned to one of the five meaningful classes after manual inspection. Second, Other does not represent a coherent semantic category; meaning emails that cannot be easily assigned to relevant subjects, are in this class. therefore, among these emails, there are no common features to bound them together so training a model based on this class only works as a fallback that absorbs everything and depressing precision on the real classes. Third, the class was very small and heavily imbalanced this could lead to precision reduction for the real categories (Figure 1). For these reasons, emails in this grouped were either reassigned or removed.

Priority labels went through the same agreement-based pipeline, but a major design question was to keep the three original categories, High, Medium, and Low, or collapse the task into a binary classification problem.

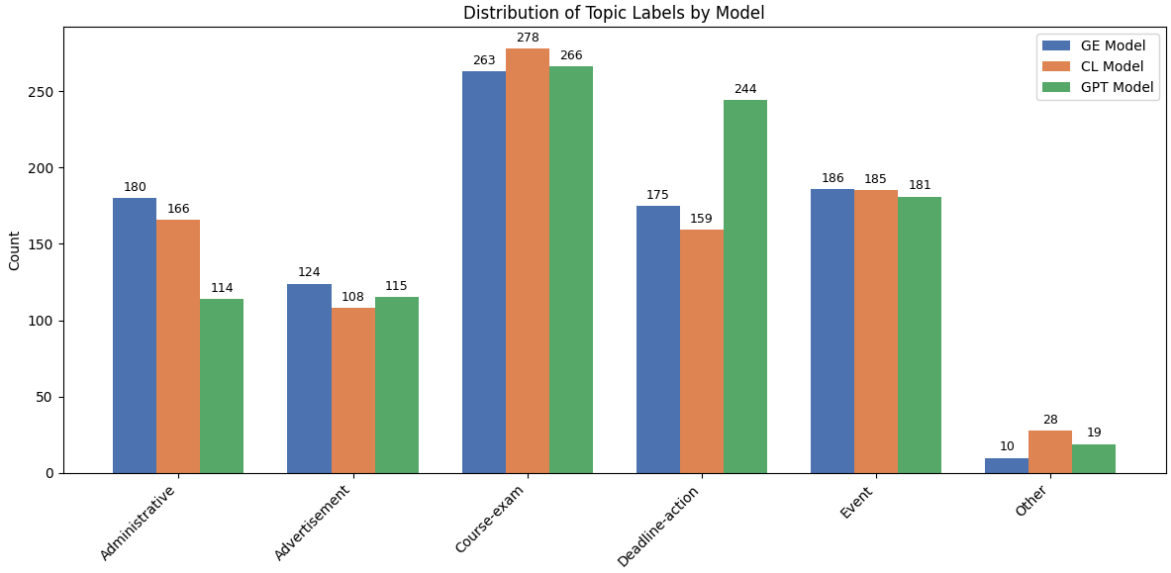

Figure 1: Distribution of Topic Labels by Model before final cleaning

Priority is inherently subjective. Medium class works as a bucket where annotators place emails that are clearly not urgent but not ignorable either. There were two approaches to create a binary classification: either preserve the High signal and lose the difference between informational and casual emails (High versus not-High). Or keep Low signal but merge urgent and merely interesting emails (Low versus not-Low). Both looked attractive because they would simplify modelling (macro-F1 increases and binary problem is easier on the same data rather than 3-class one).

After review, I decided to proceeded with three priority classes due to several reasons. First, a triage system is fundamentally three valued. “Act now / read later / ignore” is the natural

triage axis, therefore collapsing it not only changes the distinction the project tries to make but also changes a major goal of the proposal. Second, the data supports three classes, it is nearly uniform and all models have agreement on the general pattern (Figure 2). Finally, although the Medium class is ambiguous, it is informative rather than destructive. It reflects real uncertainty of email triage and can later be used to analyse model errors carefully.
There are still drawbacks to keeping three classes. In particular, the Macro-F1 score is expected to be lower, and Medium remains the largest source of errors across all models. However, these costs are acceptable because the alternative would remove the central distinction between urgent, relevant, and low-impact emails.

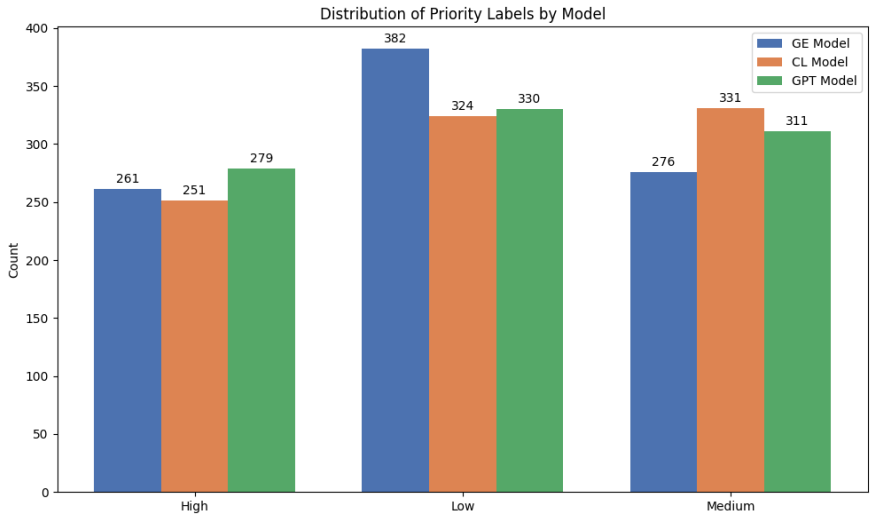

Figure 2: Distribution of Priority Labels by Model

As shown in Figure 2, the three models produced broadly similar priority distributions, which supports keeping the original three-class schema.

5 Preprocessing and Exploratory Data Analysis

5.1 Preprocessing

After unifying three annotators' dataset and cleaning them, the result is saved as gold_dataset which is the input of the preprocessing notebook. First of all, several integrity checks are performed. Including checking: number of rows, invalid topic or priority label, duplicate ID, and empty subjects and bodies. In the end, the dataset contained 919 rows, with no duplicate IDs. Although some emails had empty bodies, all emails had non-empty subject, so they were still useful for classification.
The text cleaning was intentionally light because the corpus is multilingual and downstream models need both vocabularies intact. The pipeline lowercased the text, removed simple HTML artifacts, replaced URLs with URL token and email addresses with EMAIL token, collapsed repeated whitespace, and trimmed the final text. However, it did not remove punctuations, digits, stopwords, or apply stemming and lemmatization. This choice was

deliberate because punctuations, numbers, dates, course codes, and bilingual function words can all carry useful information.

Finally, the pipeline creates three text outputs for modelling: text_subjects, text_body, and text_subject_body. This is useful because some emails contain strong cues in the subject, while other need body to determine the correct topic or priority. The final output is saved as frozen_dataset which is the input for EDA, and all of the models trained later.

5.2 EDA Findings

After preprocessing, I performed exploratory data analysis to better understand the structure of the frozen dataset before training any model. The goal of this step is to identify possible risks that could affect evaluation, such as repeated templates or strong dependencies between topic and priority.

As shown in Figure 3, the topic classification is slightly imbalance with course-exam being the largest class corresponding to 29.1% of the dataset which is an anticipated behavior considering it is a university mailbox. The smallest class is advertisement, with 11.6%. the other three class are distributed more evenly.

The priority task has more balance distribution as shown in Figure 4. This is useful for modeling because no class is too rare. However, it does not mean the task is easy. in fact, the medium class remains conceptually difficult to model since it represents emails that are useful but not urgent.

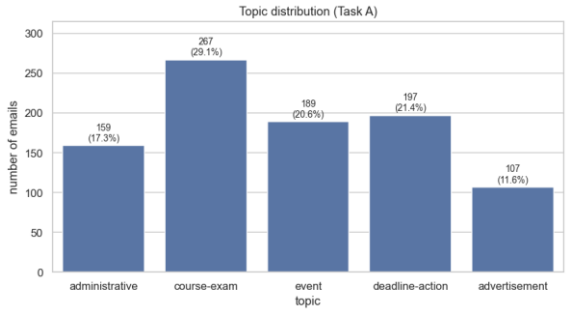

Figure 3: Topic distribution

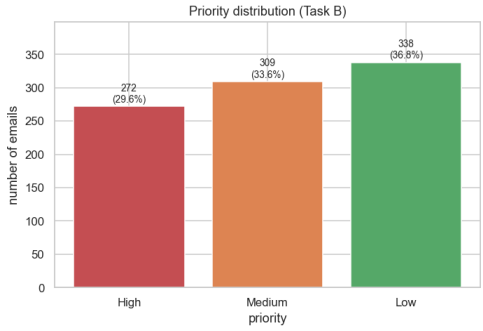

Figure 4: Priority distribution

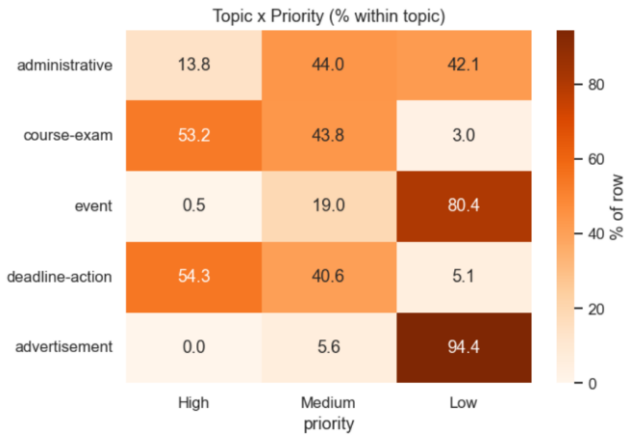

Figure 5: Topic-Priority Relationship

The topic priority heatmap (figure 5) indicates both tasks are strongly related, but not identical. For example most advertisement emails are low priority. In contrast, course-exam and deadline-action emails are much more likely to be high priority. This confirms the intuition that the topic of an email gives strong clue about its importance. However, the two tasks still should be modeled separately, since the same topic can appear with different level of urgency.
The boxplots by topic and priority (figure 6) show that length can provide some signals, but it is not enough by itself. For example course-exam emails tend to be shorter that some other categories, while other classes have longer bodies. For priority, High emails tend to be shorter on average, while Medium and Low are often longer. This suggest length may help as an auxiliary feature, but it cannot be replaced with textual understanding. A short email is not always urgent neither a long one a low priority.

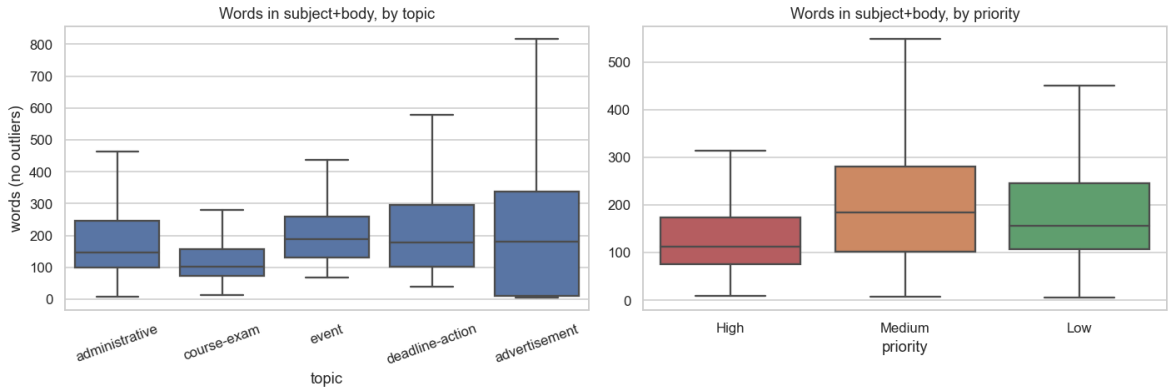

Figure 6: Words in subject + body by Topic & Priority

Another important finding is the presence of repeated subject templates. The EDA shows 22.4% of emails belong to groups with repeated cleaned subjects. This creates a risk of overly optimistic evaluation: if the same subject is template appears in both training and test sets, the model may memorize the template instead of learning general patterns. For this reason, I decided to not only report the stratified split, but also a grouped-by-subject split, where emails with the same subject are kept together. This makes the evaluation more realistic.
In conclusion, the EDA suggests three main conclusions. First, topic classification is likely easier than priority because topic labels have clearer lexical patterns. Second, priority classification is more contextual and depends on urgency, deadlines. Third, the evaluations must control for repeated templates, otherwise the results may overestimate the true performance of the models.

6 Methodology and Models

6.1 Text-Only Baseline

The purpose of this step is to establish a reliable baseline before adding more complex features. This is crucial since in small dataset a simple classical model might perform surprisingly well, especially when the labels are strongly connected to the vocabulary of the emails.

For this baseline, I used TF-IDF representations of the email text combined with two linear classifiers: Logistic Regression and Linear SVM [4]. TF-IDF was chosen because it is simple, and well suited for small text classifications. It represents each email according to the importance of its words and short phrase, while reducing the influence of common terms. For its hyperparameters, I set the vectorizer n-gram range to (1-2) to put the focus on bigrams to capture phrases that single tokens lose. Both tasks get 735 emails as train and 184 emails as test. Every input field for that task is then evaluated on the same indices.

I tested three different input types: text_subject, text_body, and text_subject_body. This comparison was useful because the subject and the body provide different kind of information. The subject is short but often contains strong topic cues. The body is longer and contains more context which is essential for priority classification.

	Topic			*Priority*			
Model	Input field	CV macroF1	Test acc	Test macro F1	CV macroF1	Test acc	Test macro F1
LogReg	text_subject	0.739 ± 0.035	0.783	0.774	0.704 ± 0.017	0.712	0.702
LogReg	text_body	0.800 ± 0.027	0.821	0.812	0.722 ± 0.022	0.734	0.729
LogReg	text_subject_body	0.820 ± 0.022	0.832	0.824	0.726 ± 0.032	0.734	0.729
LinSVC	text_subject	0.760 ± 0.030	0.815	0.799	0.703 ± 0.020	0.734	0.726
LinSVC	text_body	0.824 ± 0.039	0.859	0.846	0.737 ± 0.033	0.766	0.765
LinSVC	**text_subject_body**	**0.844 ± 0.035**	**0.870**	**0.861**	**0.739 ± 0.036**	**0.761**	**0.759**

Table 2: TF-IDF + (LOGREG & LINSVC) baseline result

The results in table 2 show that linear SVM performed better than Logistic Regression for both tasks. For topic classification, the best text-only model used text_subject_body and reached macro-F1 score of 0.861. For priority, the best model reached a macro-F1 score of 0.765. Interestingly although Priority has only 3 classes compared to topic which has 5, it achieved lower score which suggest priority is harder.

However, when the same best models were tested using a grouped-by-subject split, the performance dropped. Topic macro-F1 from 0.861 to 0.804, and priority from 0.765 to 0.705. This drop shows that part of the performance comes from the repeated patterns appearing in both training and test sets. Therefore, the grouped-by-subject gives amore realistic estimate of how well the model generalizes to new email templates.

6.2 Text Feature Diagnostics

After creating the baseline for the experiment, I aimed to understand where the performance of baseline is coming from since it achieved relatively strong results. The goal of this step was to check whether the models were leaning meaningful textual patterns or mapping simple shortcuts such as email length or language identity.

The first diagnostic tested length, language proxy, and their combination. I chose these features, because long emails tend to be announcements or events, while short emails may

be urgent messages or notifications. Similarly, English and Italian are not distributed evenly across all categories. However, these features alone should not be enough to solve classification task.

The results in Table 3 show that using only length and language features, models can reach a limited performance compared to TF-IDF. It indicates that baseline models were not simply exploiting superficial properties. This was an important validation since it made the later modelling results more trustworthy.

feature set	topic macroF1	priority macroF1
length only	0.341	0.406
language only	0.330	0.457
length + language	0.379	0.462

Table 3:shortcut baselines - stratified best of LogReg / LinSVC

The second diagnostic tested different TF-IDF representations. In addition to the unigram and bigram features, I tested character n-gram and combined word-character representations. Character n-grams can sometimes be useful in multilingual text because they capture word fragments and morphology. This seemed useful in multilingual dataset.

The results in Table 4 show an important difference between two tasks. For topic, character n-gram slightly improved the optimistic stratified score, specially for categories with repeated language. However, this improvement did not show up in the grouped-by-subject evaluation. When repeated subject templates were separated between training and test sets, the character-based model performed worse than original word-level TF-IDF baseline. This suggests that character n-grams were learning template-specific patterns rather than more general semantic information.

variant	input	macroF1	n_features
word_1gram	text_subject_body	0.853	6 336
word_1_2gram (baseline)	text_subject_body	0.861	23 341
Char_3_6gram	Text_subject_body	0.863	71 206
word_1_2 + char_3_5	text_subject_body	0.866	70 780
char_3_5gram	text_body	0.866	46 769
char_3_6gram	text_body	0.860	70 169

Table 4:Topic n-gram variants

For priority based on Table 5, changing the TF-IDF representation did not help. This supports the initial observation where the priority is not a vocabulary representation problem. An email's urgency strongly depends on the context, not the form of the words.

variant	input	macroF1	n_features
word_1_2gram (03)	text_body	0.765	22 603
char_3_6gram	text_body	0.730	70 043
word_1gram	text_subject_body	0.755	6 302
char_3_6gram	text_subject_body	0.730	71 251

Table 5:Priority n-gram variants

The main conclusion of this diagnostic is that the original word-level TF-IDF baseline should remain the backbone for further experiments. Although testing character n-grams were interesting, but they increased the risk of template memorization.

6.3 Hybrid Feature Engineering

In this stage, I added hybrid features on the top of the TF-IDF representation. The motivation was that some useful information in the emails such as dates and attachment files contain signals that can improve the model's understanding of the role and importance of an email.
Feature groups that I considered including: text length, language proxy, metadata, regex-based signals, and small bilingual lexicons. The language proxy estimates whether an email is mostly Italian or English. Metadata features included sender domain, date and time information, attachment count, number of recipients, and CC information. These signals are practical because university emails often follow repeated institutional or mailing-list patterns.
Regex-based features used to capture structural cues in the raw subject and body, such as dates, times, reply prefixes, bracket tags, punctuations, and capitalization. These patterns are important because some relevant information appear as format rather than vocabulary. For example, date can be a strong cue for a deadline email. Finally, some bilingual lexicons were added for urgency, academic content, events, and advertisements, using both Italian and English keywords.

task	split	text-only	all-hybrid	Δ
topic	stratified	0.861	0.854	−0.007
topic	grouped-by-subject	0.804	0.853	+0.049
priority	stratified	0.765	0.747	−0.018
priority	grouped-by-subject	0.683	0.739	+0.056

Table 6: Hybrid models results

Based on Table 6, the most interesting result was that hybrid features did not simply improve the optimistic stratified score. In fact, the stratified score slightly decreased compared with the baseline. However, the grouped-by-subject score improved clearly. This means that hybrid features helped the model generalize better to unseen email templates, instead of relying too much on repeated subjects. For topic and priority, the macro-F1 improved more than 5%.

This is an important achievement because it changes how the hybrid model should be interpreted. The hybrid features were not mainly useful because they increased the easiest score; they were useful because they made model more robust. They encouraged the classifier to use more general signals such as dates, language, and urgency cues instead of relying on templates.

6.4 Transformer Fine-Tuning

After the classical and hybrid models, I also tested transformer-based models [2] to see whether pretrained models can much achieve better results or not. This step was especially crucial for priority classification, because previous experiments showed that priority is harder to solve using TF-IDF alone. Unlike topic which is basically mapping vocabulary to classes, priority depends more on context, urgency, and the practical meaning of the email.
I compared two transformer backbones: XLM-RoBERTa [3], a general multilingual model, and Umberto, an Italian-tuned transformer (Table 7). This comparison was useful because the dataset contains both Italian and English corpus, but Italian is the dominant one. The goal was to understand whether a broad bilingual model [5] or a language-specific model would work better for this type of dataset.

backbone	size	training corpus
xlm-roberta-base	125M	100 languages, CC100
umberto	110M	Italian CommonCrawl

Table 7: Transformers Configurations

The models were fine-tuned separately for the two tasks. For topic, the input contained both subject and body, since topic information can appear both in subject and in the body. For priority however, the body was more important, because urgency and required actions are usually explained in the main content of the email rather than only in the subject.
The training was done on RTX 3060 (6 GB), with hyperparameters set as follows: maximum sequential length 256 to cover ~90% of bodies, batch size 16, optimizer AdamW with 2×10^{-5} learning rate, also since the dataset is small no more than 5 epoch was needed.

task	backbone	stratified macroF1	grouped macroF1
topic	xlm-r	0.853	0.783
topic	umberto	0.858	0.815
priority	xlm-r	0.761	0.744
priority	umberto	0.752	0.803

Table 8: Transformer's macro-F1 stratified and grouped by subject

Results in Table 8 showed that Umberto performed better than XLM-RoBERTa on the grouped-by-subject evaluation for both tasks. This indicates that Italian-specific pretraining was useful for this dataset, since many administrative emails are written in Italian. For topic classification however, the transformers did not outperform the hybrid model. The hybrid model remained stronger on the honest grouped split, which confirms the topic is already well handled by TF-IDF and hybrid features.

The most important improvement appeared in priority classification. Umberto achieved a grouped macro-F1 of about 0.803, drastically higher than hybrid model (0.739). This was the strongest results for the priority in this project. It suggests that transformers are better in capturing the contextual meaning of urgency, especially around the difficult Medium class.

7 Results and Discussion

7.1 Topic Classification Results

For topic classification, the text only TF-IDF was already strong. The best model was TF-IDF with Linear SVM using the combined subject and body. It achieved a macro-F1 of 0.861 on the stratified and 0.804 on the grouped-by-subject split. The gap between them confirms that some of the performance came from repeated email templates.

The hybrid model produced the best overall results under honest grouped evaluation. Although its stratified score was slightly lower than baseline, its grouped score achieved 0.853. This means that hybrid features helped the model to generalize better. Features such as sender domain, date, metadata, regex patters, and small lexicon made the classifier less dependent on memorized patterns.

The transformer models specially Umberto, were competitive yet not able to outperform hybrid methods. Since topic labels are strongly connected to visible vocabulary, a well-designed TF-IDF and hybrid pattern system is enough to capture most of useful signals.

Model	Best input/features	Stratified Macro-F1	Grouped Macro-F1
TF-IDF + Linear SVM	text_subject_body	0.861	0.804
Hybrid + Linear SVM	**Text + length + language + metadata + regex + lexicon**	**0.854**	**0.853**
Umberto Transformer	text_subject_body	0.858	0.815

Table 9: Topic classification scores of top models

As shown in the table 9, the hybrid model gives the best grouped-by-subject result for topic classification, this supports the idea of that topic benefits from both lexical and small contextual signals, but does not necessarily require transformer model.

7.2 Priority Classification Results

Priority classification was more difficult. The best text-only baseline achieved 0.765 macro-F1 on the stratified split, but its group performance was much lower. This indicates that text alone was not enough to reliably predict priority when repeated templates were removed from evaluation.

The hybrid model improved the grouped score to about 0.739. This improvement is important because priority depends on more than vocabulary. Metadata, sender information, regex, and urgency lexicon all participated in increasing the score. However, the hybrid model still struggles with Medium class, which is the most subjective priority category.

The best priority result came from Umberto transformer. It achieved the grouped macro-F1 of 0.803, clearly outperforming the hybrid model. This suggests the transformer representations are more useful for more contextual tasks such as priority rather than topic. Unlike TF-IDF, the transformer can better capture the surrounding context of urgency, such as whether an email requires an immediate action or is generally relevant.

Model	Best input/features	Stratified Macro-F1	Grouped Macro-F1
TF-IDF + Linear SVM	text_body	0.765	0.705
Hybrid + Linear SVM	Text + length + language + metadata + regex + lexicon	0.747	0.739
Umberto Transformer	**text_body**	**0.752**	**0.803**

Table 10: Priority classification scores of top models

As shown in Table 10, the transformer gives the best results for priority classification, this confirms that priority is less lexical and more contextual than topic classification.

7.3 Error Analysis

The error analysis confirms the difference between two tasks. For topics, more errors happen between semantically close categories. The most confusion happens between administrative and deadline-action (Figure 7). This is expected because many university emails are both procedural and action-oriented. For example, an official university email may look administrative, but if the main point is submitting a document before deadline, it will become closer to deadline-action.

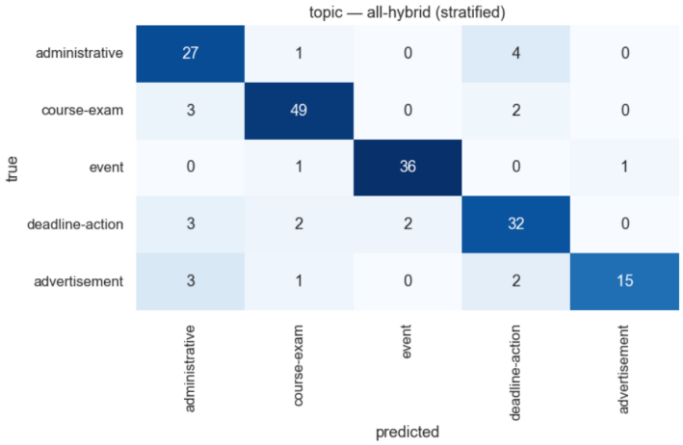

Figure 7: Topic classification confusion matrix (hybrid model)

Another recuring confusion involves course-exam and deadline-action. Some course emails include exam registration deadlines, while the others mention course-related information. These cases are difficult because the correct label depends on the dominant purpose of the email, not only presence of the words such as “exam”, “registration”, or “deadline”.

For priority classification, most errors involve class Medium (Figure 8). This is not a surprise since this class works as a boundary between urgent emails and non-urgent ones. Many mistakes are only one step away, such as High being predicted Medium or Low Predicted as Medium. This means the models usually learn the correct priority direction, yet struggle with the exact extend of them.

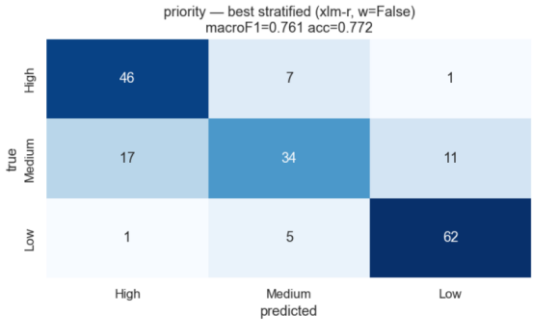

Figure 8: Priority classification confusion matrix Transformer

Overall, the results show that no single model is suited for everything. The hybrid models work best with Topic classification because they combine strong lexical performance with better generalization. On the other hand, for Priority classification, since we are dealing with context and intention which can be relative to time, and sender, an attention-based model works better to extract the true pragmatics.

8 MPV Telegram Bot

In addition to the modelling experiment, I developed an MVP Telegram bot to demonstrate how classifier can be used in practice. The motivation behind this MPV was my personal need to constantly check my university emails and since Telegram has the highest usage, integrating triage with a bot seemed like the most practical choice.

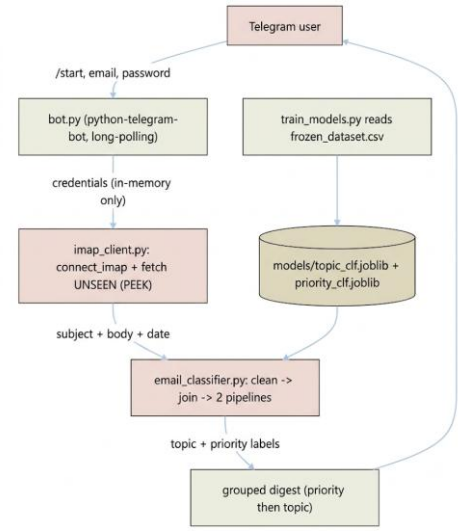

Figure 9: Telegram Bot MVP architecture diagram

8.1 Architecture

The bot follows a simple pipeline (Figure 9). First, the user interacts with Telegram bot and provides UniGe email credentials. Then, the bot connects to the mail box through IMAP and fetches emails from inbox. For each email, it extracts the subject, body, sender, and date, and applies the same preprocessing format during training. The trained classifiers predict both priority and topic of each email.

8.2 Basic Usage instructions

The usage is simple. The user starts the bot in Telegram, enters UniGe email and password, and the bot scans the unread inbox messages. After classification, it returns the summary organized by priority and topic. High priority emails are shown first, followed by medium and low priority in each group then the emails are organized based on their topic.

Since this is only an MVP, credentials are used only during the scan and are not stored permanently. A production version would need a safer authentication mechanism.

9 Limitations and Future Work

The most important limitation of this project is its dataset. It is small and personalized for a single student so any model trained on this dataset might not capture the subtle difference of other students' mailbox. Second, the labels were created with LLM assistance and manual review, but some ambiguity remains, especially for Medium class. In many cases the difference between High and Medium depends on the deadline distance or personal relevance, which is difficult to infer from.

Another limitation is that the current models do not explicitly compute the number of days until a deadline. The regex features can detect dates, but do not fully interpret them in relation to the email date. Feature work could add a deadline distance feature, which could improve priority classification.

The MVP Telegram bot is also only a prototype. It demonstrates the practical usage of an email triage, but a real deployment requires better authentication, tighter security, and robust email management.

10 Conclusion

This project developed an end-to-end multilingual email triage system. Starting from real UniGe emails, the work included email extraction, LLM-assisted annotation, manual cleaning, preprocessing, exploratory analysis, classical baseline, hybrid feature engineering, transformer fine-tuning, and Telegram bot prototyping.

The results indicates that topic classification can be handled effectively with classical and hybrid NLP methods, in contrasts, priority classification is harder and more contextual, thus the best priority classification result was reached using fine-tuning transformer Umberto.

In conclusion this project proves that a practical email triage system can be built with a combination of careful dataset construction and interpretable models.

References

[1] Jardim, B., Rei, R. and Almeida, M. S. C. 2021. Multilingual Email Zoning. In Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop, pp. 88-95.

[2] Reimers, N. and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP).

[3] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L. and Stoyanov, V. 2020. Unsupervised Cross-lingual Representation Learning at Scale. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 8440-8451.

[4] Borg, A., Boldt, M., Rosander, O., & Ahlstrand, J. (2020). E-mail classification with machine learning and word embeddings for improved customer support. Neural Computing and Applications, 33, 1881-1902. https://doi.org/10.1007/s00521-020-05058-4

[5] Jiang, X., Liang, Y., Chen, W., & Duan, N. (2022). XLM-K: Improving Cross-Lingual Language Model Pre-training with Multilingual Knowledge. Proceedings of the AAAI Conference on Artificial Intelligence, 36, 10840-10848. https://doi.org/10.1609/aaai.v36i10.21330